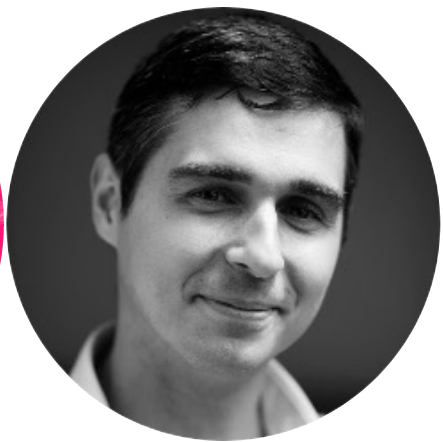


Diving into Pattern Matching



Andrejs Doronins

Software Developer in Test



```
Object obj = 5;
```

```
if (obj instanceof Integer i) {  
    System.out.println("Integer value: " + i);  
}
```



Without Pattern Matching

```
// Reflection, Deserialization, Legacy  
Object obj = getThing();
```

```
if (obj instanceof Integer) {  
    Integer i = (Integer) obj; // Explicit cast  
  
    System.out.println(i.intValue());  
}
```



With Pattern Matching

```
Object obj = getThing();
```

Pattern variable



```
if (obj instanceof Integer i) {
```

```
    System.out.println(i.intValue());
```

```
}
```



With Pattern Matching

```
Object obj = getThing();
```

```
if (obj instanceof Integer) {
```

```
    // do thing without the variable
```

```
}
```



Flow Scoping

Number n = 123;

```
// n must be int, and so    -> type is known on this side  
if (n instanceof Integer i && i.compareTo(5) > 0)  
    System.out.print(i);  
}
```

```
// even if n isn't an int    -> do this anyway (type unknown)  
if (n instanceof Integer i || i.compareTo(5) > 0)  
    System.out.print(i);  
}
```



Flow Scoping

```
Number number = 145;
```

```
if (number instanceof Integer i)  
    System.out.print(i.intValue());
```

```
System.out.print(i.intValue());
```



Flow Scoping

```
Number number = null;
```

```
if (number instanceof Integer i) // false  
    System.out.print(i.intValue()); // will not run
```

```
// boom!  
System.out.print(i.intValue());
```



Question 1

What will this code output?

```
class Animal {}  
class Dog extends Animal {}  
class Cat extends Animal {}  
  
Animal pet = new Cat();  
  
if (pet instanceof Dog d)  
    System.out.println("Dog");  
else if (pet instanceof Cat t)  
    System.out.println("Cat");  
else  
    System.out.println("Animal");
```

Cat



Question 2

What is the result of this code?

```
String getType(Object obj) {  
    if (obj instanceof Number n && n != null)  
        return "Num";  
  
    if (obj instanceof String c &&  
        !c.isEmpty() || c != null)  
        return "String";  
  
    return "Unknown";  
}
```

Compilation error
(usage of ||)



Question 3

What will this code output?

```
Object obj = "Hello";

if (obj instanceof String s)
    System.out.println("Length: " + s.length());

else if (obj instanceof Integer)
    System.out.println("Value: " + i * 2);

System.out.println(s.toUpperCase());
```

2 compilation errors



Question 4

What happens if “checkNumber(1);” is invoked?

```
void checkNumber(int num) {  
    var result = switch (num) {  
        case Integer i -> "Integer: " + i;  
        case 0 -> "Zero";  
        default -> "Other number";  
    };  
    System.out.println(result);  
}
```

Compilation error

(int instead of
Integer)



Question 4.b

What happens if “checkNumber(1);” is invoked?

```
void checkNumber(Integer num) {  
    var result = switch (num) {  
        case Integer i -> "Integer: " + i;  
        case 0 -> "Zero";  
        default -> "Other number";  
    };  
    System.out.println(result);  
}
```

Compilation error

(“case Integer”
dominates “case 0”)



Question 4.c

What happens if “checkNumber(1);” is invoked?

```
void checkNumber(Integer num) {  
    var result = switch (num) {  
        case Integer i -> "Integer: " + i;  
        default -> "Other number";  
    };  
    System.out.println(result);  
}
```

Compilation error

(“case Integer”
still covers the
entire range)



Question 5

What happens if “makeMovement(null);” is invoked?

```
void makeMovement(Object o) {  
    var output = switch (o) {  
        case Object obj -> "object";  
        default -> "default";  
        case null -> "null";  
    };  
    System.out.println(output);  
}
```

Compilation error

(“case default”
dominates “case
null”)



Question 5.b

What happens if “makeMovement(null);” is invoked?

```
void makeMovement(Object o) {  
    var output = switch (o) {  
        case Object obj -> "object";  
        case null -> "null";  
        default -> "default";  
    };  
    System.out.println(output);  
}
```

Compilation error

(“case Object”
dominates)



Question 6

What will this code output?

```
String status = "Closed";

int code = switch (status) {
    case String s when s.equals("Open") -> 10;
    case Object s when !"".equals(s) -> 20;
    case null -> {yield 30;}
    default -> 40;
};

System.out.print(code);
```

20



Question 7

What will this code output?

```
enum DoorState { OPEN, CLOSED, LOCKED }

DoorState state = DoorState.CLOSED;
int code = switch (state) {
    case DoorState s when s == DoorState.OPEN -> 10;
    case null -> 20;
    case default -> {
        throw new RuntimeException("Boom!");
    }
};
System.out.print(code);
```

Compilation error

Invalid syntax:
"case default"

